

plano tecnico execucao

Plano de Execução — Central de Recorrências (Pix Automático)

Contexto

- Épico de Origem: Funcionalidade de Pix Automático
- Histórias Incluídas: NAI-58, NAI-59, NAI-60, NAI-61
- Repositório: samuelpimentel4sys/demo-bradesco — branch base: main

Arquitetura do Repositório (Resumo)

- Stack: Monorepo com Frontend (Angular 21) e Backend (FastAPI).
- Camadas e Convenções (Frontend): A aplicação Angular é modularizada por feature. Cada funcionalidade principal reside em seu próprio diretório dentro de frontend/src/app (ex: login/, dashboard/). Os serviços compartilhados ficam em frontend/src/app/services/. As rotas são definidas em frontend/src/app/app.routes.ts e protegidas por guardas em frontend/src/app/guards/.

Padrão de Referência

- Módulo usado como modelo: frontend/src/app/dashboard/
- Por que se aplica: O dashboard é uma tela segura (acessada após login) que exibe informações para o usuário, um padrão idêntico ao da nova Central de Recorrências. Seguiremos a mesma estrutura de componente, rota e estilo.

Mapa de Mudanças por História

História	Arquivos existentes afetados	Arquivos novos a criar	Camada	Padrão de referência usado	Contratos de API envolvidos	Dependências
----------	------------------------------	------------------------	--------	----------------------------	-----------------------------	--------------

NAI-59	(Nenhum)	backend/main.py (adição de endpoint)	Backend	N/A	GET /api/recorrencias (novo)	Nenhuma
NAI-58	frontend/src/app/recorrencias/recorrencias.css frontend/src/app/services/recorrencias.service.ts	frontend/src/app/recorrencias/recorrencias.html frontend/src/app/recorrencias/recorrencias.ts	Frontend	Frontend	GET /dashboard GET /api/recorrencias	NAI-59
NAI-60	(Nenhum)	frontend/src/app/recorrencias/recorrencias.ts	Frontend	Frontend	frontend/src/app/dashboard/dashboard.html	NAI-58
NAI-61	frontend/src/app/services/recorrencias/recorrencias.html frontend/src/app/recorrencias/recorrencias.ts	frontend/src/app/services/recorrencias/recorrencias.html frontend/src/app/recorrencias/recorrencias.ts	Frontend	Frontend	POST /api/recorrencias/{id}/approve POST /api/recorrencias/{id}/pause POST /api/recorrencias/{id}/cancel (novos)	NAI-60

Ordem de Implementação

A implementação deve seguir a ordem abaixo para respeitar as dependências entre backend e frontend.

Criar o Endpoint de Consulta (NAI-59)

- Arquivo:** backend/main.py
- Ação:** Adicionar um novo endpoint GET /api/recorrencias que retorna uma lista mockada de autorizações de Pix Automático. Isso desbloqueia o desenvolvimento do frontend.

Contrato de Retorno (Exemplo):

```
[
  {"id": 1, "beneficiario": "Empresa A", "valor": 100.00, "status": "ativa"},
  {"id": 2, "beneficiario": "Empresa B", "valor": 50.00, "status": "pausada"},
  {"id": 3, "beneficiario": "Empresa C", "valor": 75.00, "status": "pendente_aprovacao"}
]
```

Criar o Módulo e Serviço da Central (NAI-58)

Ação 1: Criar o Serviço:

- Arquivo:** frontend/src/app/services/recorrencias.service.ts (novo)

- **Conteúdo:** Criar um serviço Angular com um método `getRecorrencias()` que faz uma requisição HTTP GET para o endpoint `/api/recorrencias` criado no passo 1.

Ação 2: Criar a Estrutura do Componente:

- **Arquivos:** Criar o diretório `frontend/src/app/recorrencias/` e, dentro dele, os arquivos `recorrencias.ts`, `recorrencias.html`, e `recorrencias.css`, seguindo o modelo do componente `dashboard`.
- **Conteúdo:** No `recorrencias.ts`, injetar o `RecorrenciasService` e criar a lógica para gerenciar os estados de `loading`, `error` e `success`. O template `recorrencias.html` deve exibir um spinner durante o `loading` e uma mensagem de erro caso a requisição falhe.

Ação 3: Adicionar a Rota:

- **Arquivo:** `frontend/src/app/app.routes.ts`
- **Conteúdo:** Adicionar uma nova rota `/recorrencias` que renderiza o `RecorrenciasComponent` e é protegida pelo `AuthGuard`.

Exibir a Lista de Autorizações (NAI-60)

- **Arquivos:** `frontend/src/app/recorrencias/recorrencias.html` e `recorrencias.ts`
- **Ação:** Com base no passo 2, implementar a lógica para, em caso de sucesso, iterar sobre a lista de recorrências retornada pela API e exibi-las em formato de cards. Implementar também a exibição de uma mensagem de "Nenhuma autorização encontrada" quando a lista estiver vazia.

Implementar as Ações de Gerenciamento (NAI-61)

Ação 1: Adicionar Endpoints no Backend:

- **Arquivo:** `backend/main.py`
- **Conteúdo:** Adicionar os endpoints POST para as ações (`/api/recorrencias/{id}/approve`, `/pause`, `/cancel`). Inicialmente, podem apenas retornar um status de sucesso.

Ação 2: Adicionar Métodos no Serviço Frontend:

- **Arquivo:** `frontend/src/app/services/recorrencias.service.ts`
- **Conteúdo:** Adicionar métodos como `aprovarRecorrencia(id)`, `pausarRecorrencia(id)`, etc., que chamam os respectivos endpoints no backend.

Ação 3: Adicionar Botões na UI:

- **Arquivo:** `frontend/src/app/recorrencias/recorrencias.html`
- **Conteúdo:** Em cada card de recorrência, adicionar os botões de ação (Aprovar, Pausar, Cancelar, etc.) e vinculá-los aos métodos correspondentes no `recorrencias.ts`, que por sua vez chamarão os métodos do serviço.

Riscos Técnicos e Mitigação

Risco: O serviço de autenticação (`auth.service.ts`) é compartilhado. Qualquer alteração acidental pode impactar o Login e o Dashboard.

- **Mitigação:** As novas alterações estão bem isoladas no `recorrencias.service.ts`. O time deve garantir que nenhuma alteração seja feita nos serviços existentes, a menos que seja estritamente necessário e validado.

Risco: Os contratos de API para as ações (aprovar, pausar) ainda não existem.

- **Mitigação:** O plano propõe a criação de endpoints mockados no backend (`main.py`) para não bloquear o desenvolvimento do frontend. O contrato final deverá ser alinhado entre as equipes, mas o desenvolvimento pode começar com base na estrutura proposta.

Bloqueios (não prontos para execução)

- **Nenhum.** Todas as histórias estão detalhadas o suficiente para iniciar a implementação, começando com dados mockados no backend para habilitar o trabalho paralelo.

Definition of Ready para o agente executor

- Todo contrato de API necessário está definido (inicialmente mockado) ou explicitamente marcado como "a confirmar".
- Nenhuma mudança depende de uma história bloqueada.
- Padrão de referência (`dashboard`) definido para a nova tela.
- Riscos de regressão mapeados e com mitigação.